

Exam 2 – Paper and Pencil part (Fall, 2020-21)

REQUIRED as part of your Admission Ticket for Exam 2

Name: _____ Section: _____

Rules and Expectations Sections 1 and 2 for Mutchler. Section 3 for Nouredine. Section 4 for Ebrahimi.

At the beginning of this exam, you will receive the **Expectations about Academic Integrity** for this exam -- it is the same as what you were given to read previously. Re-read that document as needed. **Sign it and turn it in when you finish this exam (both parts).**

Two parts (this is Part 1, Paper-and-Pencil)

For this part, the **ONLY** external resource you may use is **two 8½ by 11-inch sheets of paper** (or **one double-sided sheet**), with whatever you want on it, typed or handwritten or a combination of the two. You must have prepared the sheets *before* beginning this exam. You may also use a calculator if you like (but only for calculating).

Problem	Points Possible	Points Earned	Comments
1	7		
2	3		
3	10		
4	10		
5	8		
6	12		
Total (of 100 on the exam)	50		

Communication

For both parts of the exam, **you must not communicate with anyone** except your instructors and their assistants, if any. In particular:

- You must not talk with anyone else or exchange information with them during this exam.
- **After** this exam, **you must not talk about the exam with anyone who has not yet taken it.**

Do NOT use email, chat or the like during this exam. Close any such applications now.

1. Consider a function named ***do_it*** that takes a ***list of integers*** as its sole argument. For each of the following possible specifications for what ***do_it*** returns:

Circle **Yes** if the code for ***do_it*** would require a loop.

Circle **No** if the code for ***do_it*** would NOT require a loop.

If ***do_it*** returns:

- | | | |
|--|-----|----|
| a. The number of odd integers in the list. | Yes | No |
| b. The average of the integers in the list. | Yes | No |
| c. The number of integers in the list. | Yes | No |
| d. True if the first integer in the list is even, else False . | Yes | No |
| e. True if the list contains no even integers, else False . | Yes | No |
| f. The second smallest integer in the list. | Yes | No |
| g. The last integer in the list. | Yes | No |

2. Consider a function named ***middle*** that takes a single argument: a sequence of numbers, where the length of the sequence is *guaranteed* to be odd.

The function returns the number at the middle of the sequence. For example:

- `middle( [90, 10, 45] )` returns **10**
- `middle( [4, 1, 6, 18, 10, 12, 21] )` returns **18**
- `middle( [33] )` returns **33**

Write (in the box below) a complete implementation of the above ***middle*** function.

```
def middle(numbers):
```

3. Consider the code snippet below. It is a contrived example with poor style, but it will run without errors. What does it print when it runs? Write your answer in the box to the right of the code. Show your work by making notations in the code or by using the empty space below or on another sheet of paper, as desired.

```
def home():  
    x = bear(3, 8)  
    print("Home A:", x)  
    y = bear(ant(4), ant(1))  
    print("Home B:", y)  
  
def ant(x):  
    print("Ant:", x)  
    x = x + 5  
    return x  
  
def bear(x, y):  
    print("Bear A:", x, y)  
    r = ant(x + 4)  
    print("Bear B:", r)  
    return ant(r + y)  
    print(ant(100))  
    return 33  
  
print("OK:", ant(101) % 100)  
home()
```

Output:



Problem #4 is purposefully omitted.

Page 6

5. Consider the code snippet below. It is a contrived example with poor style, but it will run without errors.

What does it print when it runs? Write your answer in the box below.

Show your work in any way that you think would be helpful.

```
s = [5, 3, 1, 8, 4,
     9, 7, 6, 2, 10, 20]
a = 3
b = 0
print(len(s))
for k in range(1, 10, 3):
    a = a + s[k]
    b = b + s[len(s) - k - 1]
    print("A.", k, len(s) - k - 1)
    print("B.", s[k], s[len(s) - k - 1])
    print("C.", a, b)
    print()
print("Now:", a, b)
```

6. Consider a function named **average** that takes a single argument: a sequence of numbers, where the length of the sequence is *guaranteed* to be odd.

The function returns the average of the numbers in the sequence that are bigger than or equal to the number at the middle of the sequence. For example:

- `average( [1, 7, 6, 5, 10, 3, 9] )` returns **7.4**
since the number at the middle of the sequence is **5**
and the numbers in the list bigger than or equal to **5** are **7, 6, 5, 10** and **9**
and those **5** numbers add up to **7 + 6 + 5 + 10 + 9**, which is **37**,
and the average of **5** numbers that sum to **37** is **37 / 5**, which is **7.4**.
- `average( [2, 1, 4, 4, 7] )` returns **5.0** since the number at the middle of the sequence is **4**, and the numbers in the list bigger than or equal to **4** are **4, 4** and **7**, and the sum of those **3** numbers is **15**, so the average of them is **15/3**, which is **5.0**.
- `average( [9, 4, 7, 6, 2] )` returns **8.0** since the number at the middle of the sequence is **7**, and the numbers in the list bigger than or equal to **7** are **9** and **7**, and the sum of those **2** numbers is **16**, so the average of them is **16/2**, which is **8.0**.

Write (in the box below) a complete implementation of the above **average** function.

```
def average(numbers):
```